EXP NO. 1 BASIC NETWORKING IN LINUX AND WINDOWS OPERATING SYSTEMS

AIM:

To study and execute various basic networking commands in the windows and Linux operating systems to understand their purpose and output.

Basic Networking commands in Windows

commands	Description
dir	Display the files and folders in the current directory
cd	changes the current directory
cls	clears the command prompt screen.
MKdir (or) md	creates a new directory (folder)
rmdir (or) rd	Deletes an empty directory
del	Deletes one or more files.
copy	copies files from one location to another.

move	move files or folders from one location to another.
ren	Renames a file or folder
type	Displays the contents of a text file
echo	Displays a message or turns command echoing on/off.
tree	Displays a folder structure in a tree format.
ipconfig	Displays the computer's IP address and network configuration
ping	Tests the network connection to another computer or website.
tracert	Displays the path taken by data packets to a destination.
systeminfo	Displays detailed information about the computer and operating system.
tasklist	List all currently running processes
taskkill	Ends a running process using its process name or PID

Scanned with OKEN Scanner

Basic Networking commands in Linux

Commands	Description
pwd	Displays the current working directory
ls	List files and directories
cd	changes the current directory.
mkdir	Creates a new directory
rmdir	Removes an empty directory.
rm	Deletes files or directions
cp	Copies files or directories.
mv	moves or renames files or directors.
cat	Displays the content

Scanned with OKEN Scanner

Result 11/7/26

The basic networking commands were successfully executed on windows and linux terminals. The systems configuration, DNS details, routing tables, and connectivity status were obtained correctly

Exp No. 2 LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTERS

AIM:

To configure and assign an Ip address manually to a wifi-adapter in the windows operating system.

Procedure:

1. Open Control panel → Network and Internet → Network and sharing centre.
2. Click change Adapter Settings on the left side.
3. Right click on your wifi-adapter → Properties.
4. Select Internet protocol Version 4 (TCP/IPV4) and click properties.
5. Select use the following Ip address and enter:
   * Ip Address: 192.168.1.20
   * Subnet Mask: 255.255.255.0
   * Default Gateway: 192.168.1.1
   * Preferred DNS server: 8.8.8.8
6. Click ok and close all windows.
7. Open command prompt and Verify Configuration

IPconfig

Ping 192.168.1.1

Description:

Manual Ip configuration assignment for wifi allows a user to configure a static Ip address instead of using DHCP.

This ensures consistent network addressing, better device management, and improved trouble shooting in LAN/WLAN networks.

Result:

The Wifi adapter was Successfully Configured with a manual Ip address.

The computer was able to communicate with the router and access the internet Successfully.

EXPT 3: STUDY OF DIFFERENT TYPES OF NETWORK CABLES AND CRIMPING OF CABLE WITH RJ45 CONNECTOR

AIM:

To study different types of network cables and to prepare (crimp) a network cable using RJ45 connectors for establishing LAN connections between network devices

~~APPARATUS REQUIRED~~ :-

Concept:

Network cables are used to connect computers and offer networking devices for data communication the most commonly used cable in LAN is the unshielded Twisted Pair (UTP) cable. An RJ45 connector is attached to the ends of the cable of establish a network connection. The wires are arranged according to 568A or 568B wiring standard before crimping & the completed cable is tested to ensure proper connectivity.

Types of Network cables:

cable type	Description
1. UTP [Unshielded Twisted Pair]	Commonly used for Ethernet LAN's & office networks

Scanned with OKEN Scanner

2. STP [Shielded Twisted pair]		used in area with high electrical or electromagnetic Interference
3. Coaxial cable		used for cable television broadband, Internet & CCTV systems
4	optical fibre cable	used for high-speed high bandwidth & long distance communication.

Crimping procedure:

1. Cut the UTP cable to the required length.
2. Strip about 2-3 cm of the outer insulation.
3. Untwist & straighten the wires
4. Arrange the wires according to the T568B standard
5. Trim the wire ends evenly
6. Insert the wires into RJ45 connector
7. Crimp the connector using a crimping tool.
8. Test the cable using a LAN xble Tester.

Observation:

* The wires were arranged according to the T568B standard.
* The RJ45 connector was crimped successfully.
* The cable tester indicated ~~poor~~ proper continuity.
* The cable was ready for network communication.

Result:

The different types of network cables were studied and the UTP cable was successfully terminated.

Scanned with OKEN Scanner

4. IMPLEMENT PACKET SNIFFING USING RAW SOCKETS IN PYTHON

AIM: TO implement packet sniffing using raw sockets in python & capture network packets passing through network interface

Objective: TO understand how raw sockets work & how network packets can be captured & analyze using python programming

Theory:

Packet sniffing is the process of monitoring & capturing data packets travelling across a network.

. A packet sniffer intercepts packets & analyzes their contents.

Raw sockets provide direct access to low-level network protocols, allowing applications to capture packets.

In python, the socket module can be used to create raw sockets for packet sniffing.

This technique is useful for networking debugging trouble shooting & security analysis.

Algorithm:-

1. Import the required socket Module.
2. Create a raw socket
3. Bind the socket to the ~~to~~ network interface
4. Receive packets using the recvfrom()
5. display the captured packet information.
6. Repeat the process continuously to monitor network traffic.

Program:-

```
import socket:
    Sniffer = Socket.Socket(socket.
        AF_INET, SOCKET, SOCK_RAW
    SOCKET.IPPROTO_IP)
    Sniffer.bind((host, 0))

While True:
    raw_data, addr = sniffer.recv
                    (65535,

print("Packet received from:", Addr)
print(raw_data)
```

Scanned with OKEN Scanner

Output:

Packet received from: ('192.168.15', 0)
b'E\x00\x00 (...)

Packet received from: ('192.168.1.1.0', 0)
b'E,\x00\x00\x00C...

Result:

Thus packet sniffing was successfully implemented using raw sockets in python & network packets were captured and displayed.

5. DEVELOP A CUSTOMIZED PING COMMAND TO TEST THE SERVER CONNECTIVITY

Aim:

To test and verify the connectivity between the local computer and remote servers using the Ping command in windows

Description:

The ping command is a basic network utility used to check whether a host is reachable across an IP network. It measures the round-trip time for messages sent from the source to a destination and reports packet loss and latency.

procedure:

1. Basic ping command

   ping www.google.com

output:

Displays the IP address of the server, the number of packets sent, received, lost, and the average round-trip time.

2. ping localhost (to check internal network stack) command:

   Ping local host

output:

Replies from 127.0.0.1 confirm that the TCP/IP stack on the local system is working properly.

3. ping using IP Addess command:

   ping 8.8.8.8

output:

Displays connectivity and delay time (in milliseconds) between your system and Google DNS server.

4. continuous ping Test command:

   Ping www.google.com -t

output:

Continuously pings the server until stopped manually (Ctrl+C). showing live latency and Packet information.

5. ping with Specific Number of Echo Requests Command:

   Ping www.google.com -n 5

output:

sends exactly 5 echo requests and displays the result with average time and Packet loss Summary.

6. Ping with packet Size specification command:

   Ping www.google.com -l 1000

output:

sends a ping request with a packet Size of 1000 bytes. showing transmission Performance.

Scanned with OKEN Scanner

Result:

The ping commands were successfully executed in the windows command prompt. The network connectivity, latency, and packet transmission status between the local computer various servers were verified.

9. DESIGN A SIMPLE TOPOLOGY AND CONFIGURE WITH ONE ROUTER, TWO SWITCHES AND PCS USING CISCO PACKET TRACER

AIM:

To design and configure a simple network topology consisting of one router, two switches, and multiple pcs in cisco packet tracer, and enable communication between pcs.

DEFINITIONS:

* Router: A networking device that forwards data packets between computer networks, direct traffic on the internet.

* Switch: A device that connect devices within a network and uses MAC address to forward data to the correct destination.

* Pc end devices in a network that user uses interact with and receive data.

* Cables

* Straight through cable: used to connect similar devices like switch to switch.

* Console cable: used to connect pc to router or switches console port for configuration.

EQUIPMENT NEEDED:

* Cisco packet Tracer software
* 1 Router
* 2 Switches
* 4 PC's
* Approximate cables

Step-by-step procedure:

Step 1: open cisco packet tracer

* Launch cisco packet tracker on your comp.

Step 2: Add devices to workshop:

* from the bottom device toolbar, drag & drop

drop: * 1 Router
* 2 Switches
* 4 PC's (2 connected to each Switch)

Step 3: Connect devices with cables

use copper straight - Through cables to connect.

* Router Gigabit Ethernet 0/0 to switch 0 fast ethernet 0/1
* Router Gigabit Ethernet 0/1 to switch 1 fast ethernet 0/1
* PC0 to Switch 0 fast ethernet 0/2
* PC1 to Switch 0 fast ethernet 0/3
* PC2 to Switch 1 fast ethernet 0/2
* PC3 to Switch 1 fast ethernet 0/3

Step 4: Configure Router interface

* click the router.
* click tab
* Enter the IP address & give no shutdown command.

Step 5: Configure PC IP address

* click on each pc
* Desktop tab
* IP configuration & design IP addresses & Gateways.

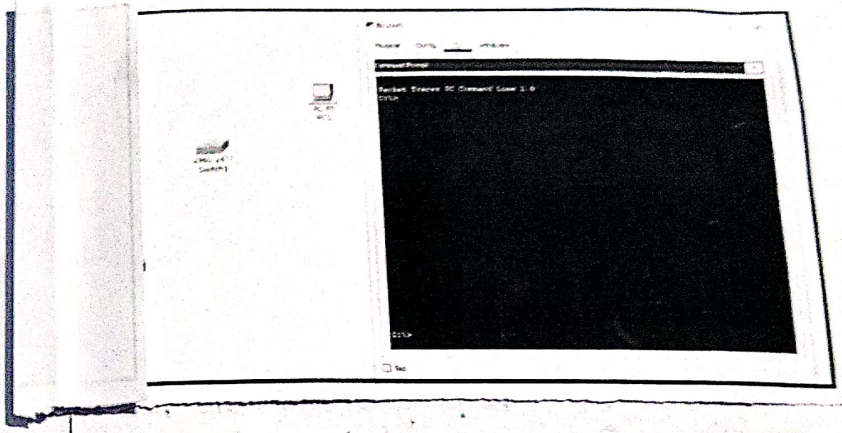
for eg:- PC0 . 192 . 168 . 1 . 10

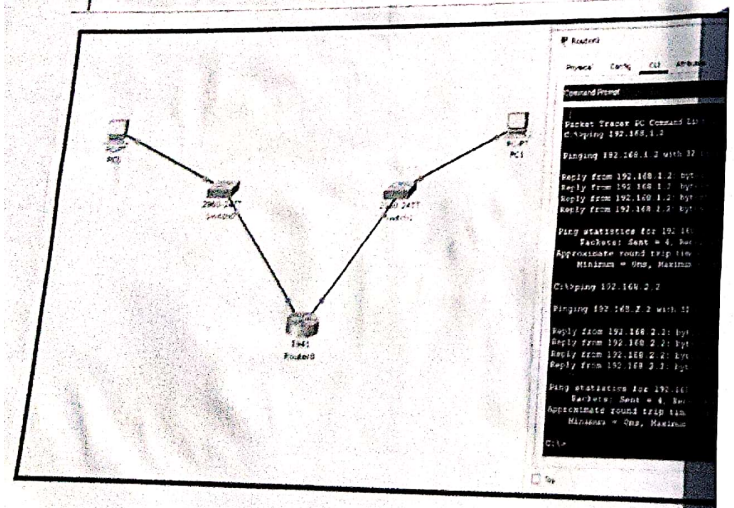
255 . 255 . 255 . 0

Scanned with OKEN Scanner

Step 6: Save and test connectivity

* Save the packet Tracer file.
* Test the connectivity using the ping command from pc0 to pc2 or pc3
* Go to CMD type ping: 192.168.2.10
* You should see replies if the network is connected successfully.





Result:

The pcs connected to different switch can communicate with each other via the router The successful pig replies Verify that the network is properly configured Operational.

10. CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER

AIM:

To learn how to add a network module to a Cisco switch, connect multiple PCs and a server, assign IP addresses, and test connectivity in Cisco packet tracer.

DESCRIPTION:

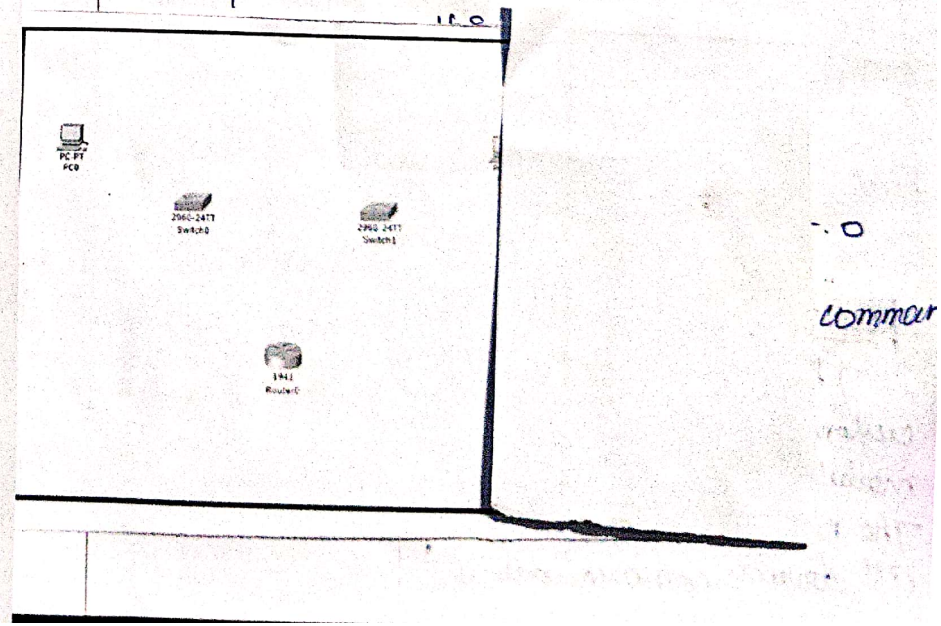
This experiment is about building a simple computer network using a switch with extra module ports. You will connect five PCs and one server, assign IPs, and check if all devices can communicate. This helps you understand how modular switches are used to create flexible networks

DEFINITION:

A Network module is an extra component you can add to a switch. It gives you more ports or adds new types of connections. In packet tracer, modules such as HWIC-4ESW let you expand the switch's capabilities.

PROCEDURE:

1. Open Cisco packet Tracer and set up your workspace.
2. Drag and drop a 2960-24TT switch to the centre.
3. Add five PCs and one server-PT from the End devices panel.
4. Power off the switch:
   - Click the switch, open the physical tab, and turn off the power button.
5. Install HWIC-4ESW module on the switch:
   - In the physical tab, drag HWIC-4ESW ~~let you expand the switch~~ into an empty module slot.
6. Power on the switch and let it boot
7. Connect all devices using copper straight-Through cables:
   - PC0 → Switch FastEthernet 0/1
   - PC1 → Switch Fast Ethernet 0/2
   - PC2 → Switch Module port 1
   - PC3 → Switch Module port 2
   - PC4 → Switch Module Port 3- Server0 → Switch module port 4 ~~Assign~~.
8. Assign IP addresses to each device:
   - PC0: 192.168.1.10
   - PC1: 192.168.1.11



Scanned with OKEN Scanner

INPUT:

Devices:

- 1 Cisco 2960-24TT Switch
- HWIC-4ESW network module
- 5 PCs (PC0 - PC4)
- 1 Server-PT (Server0)

Cables:

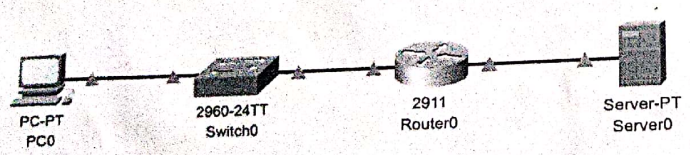
- Copper straight-through cables IP

Configuration:

- Use static IP addresses as listed above

OUTPUT:

A working network setup where every PC and the Server can successfully ping each other, proving that both built-in and module ports on the Switch are functioning.



RESULT:

The experiment demonstrates successful customization of a Switch with a network module and connectivity between all devices. The network works as expected, and all devices communicate.

7. DEVELOP A SIMPLE CALCULATOR USING XMLRPC

AIM:

TO develop a simple calculator application using XML-RPC protocol in python that performs Basic arithmetic operations like addition, Subtraction, multiplication & division through client-server communication

DESCRIPTION:

XML-RPC (extensible markup language Remote procedure call) is a protocol that uses XML to encode requests & responses & HTTP as a transport mechanism. It enables communication between a client & a server where the client invokes methods hosted on the Server, remotely.

* Server holds arithmetic function
* The client connects to the server to perform operations like addition, subtract multiplication & division.

Scanned with OKEN Scanner

Server-side code (server.py)

```
from xmlrpc.server import
                    SimpleXMLRPCServer

def add(x,y):
    return x+y
def Subtract(x,y):
    return x-y
def multiply(x,y):
    return x*y
def divide(x,y):
    if y!=0:
        return x/y
    else:
        return "Error: Division by zero"
Server = SimpleXMLRPCServer("localhost",
                    8000, log_requests=False)
Server.register_function(add,"add")
Server.register_function(Subtract,"subtract")
Server.register_function(multiply,"multiply")
Server.register_function(divide,"divide")
print("XMLRPC calculator server is
                    running on port 8000.!")
Server.serve_forever()
```

Client-side code (client.py)

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy
                    ("http://localhost:8000/")
print("---XMLRPC Calculator Test ---"):
num1, num2 = 30,10
print("Inputs: 30 and 10")
print("Addition (+):", proxy.add(num1,
                    num2))
print("Subtraction (-):",proxy.subtract(num1,
                    num2))
print("multiplication (*):"proxy.multiply(num1,num
                    2))
print("Division(/): "proxy.divide(num1,num2))
```

Software Requirements:

* OS: Window/Linux
* python version: 3.x or above
* Modules required: xmlrpc.server, xmlrpc.client (built-in with python)
* Text editor or IDE: IDLE/VS CODE/ pycharm

Scanned with OKEN Scanner

PROCEDURE :

1. Open a Python IDE or Terminal
2. Create two seperate python files : Server.py and client.py
3. Write the provided Server side code in server.py & client side code in client.py
4. Run the server.py file first to start XML-Rpc server
5. Run the client.py to perform arithmetic operations remotely.
6. Observe the results displayed on client screen & Verify the server output.

output : Server side

XML-Rpc server is running on Port 8000..

client_side :-

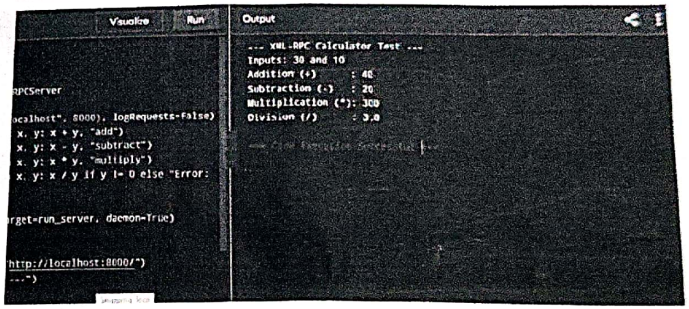
Inputs : 30 and 10

Addition : 40

Subtraction : 20

Multiplication : 300

Division : 3.0



Result : The XML-Rpc Based calculator was Successfully Implemented . The client communicated with the server using XML-Rpc Performed and arithmetic operations remotely and displayed accurate results.

Scanned with OKEN Scanner

Exp.12 NMAP TO [illegible] ARP, ICMP AND TCP/UDP PING SCAN IN TRYHACKME

Aim: Learn & Implement Nmap host discovery techniques (ARP, ICMP variants, TCP, UDP pings) to identify live hosts on a network & compare their effectiveness

Description:

Use Nmap probe types to discover which IPs are active before port / vulnerability scans covers ARP, ICMP echo, TCP syn, UDP probes & combined scans.

Definition:

Host discovery: Finding live systems PR(ARP): Layer 2 MAC resolution

PE (ICMP Echo): Standard ping

PP (ICMP Timestamp) / -PM (ICMP (mask)

PS (TCP Syn): Send syn to specified ports

Procedure:

#ARP (LAN) nmap -sn -PR 10.10.X.0/24

nmap -sn -PR 192.168.1.0/24 - packet trace. # ICMP (echo) timestamp / Mask Nmap -sn -PE 10.10.X.100

#TCP Syn/Ack (specify ports)

-PE 10.10.X.0/24 nmap -sn -PP to.(or).a

nmap -sn -PA 80, 443, 10.10.X.0/24

UDP nmap -sn -PU53, 161.10.10.X-0/24

combined / disable ARP nmap -sn -PE -PS -PA -PU 10.10.X.0/24 nmap -sn 10.10.X.0/24 - disable -arp- ping INPUT

Target IP or range (eg: 10.10.X.100 or 10.10.X.0/24)

Optional port lists For -PS, -PA, -PU (eg: 22, 80, 443, 53, 161)

Flags:- -sn, -PR -PE, -PP, -PM, -PS, -PA, -PU, -Packet trace, -disable -arp-ping

Output:

ARP

Nmap scan report For 10.10.X.1 Host is up (0.000050s latency)

MAC address : 258 IP addresses (3 hosts up)

ICMP

Nmap scan report for 10.10.X.100 host is up (0.000095s latency)

Nmap done: 1IP address (1 host up)

Result: ARP discovery reliably identified live on local subnet with lowest latency & Highest Accuracy while ICMP, TCP (Sym/Ack and UDP probes found hosts.

Scanned with OKEN Scanner

Ex:13 DEMONSTRATE

USING PCAPXRAY TOOL

AIM:-

To analyze & visualize network traffic from a PCAP file using PCAPXray tool for Forensic investigation.

steps to complete.

Step 1:- prepare your computer

make sure you have Python 3 installed

If u don't have

* Go to Python.org & download it

Step 2:- Install PCAPXray tool.

1. Open command prompt
2. Type this command & Enter
3. PIP install -PCAPXray

Step 3:- Get a PCAP file

Option A: Use a sample PCAP file

download PCAP file from website like https://wiki.wireshark.org/SampleCaptures

Step 4:- Run PCAP xray on your PCAP file

1- Put the PCAP file in Folder
2. Open command prompt
3. cd path_to_your Folder
4. Run this command
5. PCAPXray -f capture.PCAP.

Step 5:- View the Results

After command Runs, Create a file named PCAP xray.html

* protocols ports used

Step 6:- Analyse

* Look For biggest nodes
* notice unusual ports or repeated connection

Result:- PCAPXray successfully Generated a visual maps of hosts, connections & Protocols.

Ex:14 TO CAPTURE, SAVE, AND ANALYZE NETWORK TRAFFIC ON TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS USING WIRESHARK TOOL

Aim:

The aim of this experiment is to capture, save, and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS using wireshark Tool.

Procedure:

Step 1: Download and install Wireshark from its official website.

Step 2: Lauch wireshark and identify the network interface

Step 3: Before starting the capture, apply capture filters to focus on specific protocols and reduce unwanted data. Examples of capture filters you can use.

Step 4: Click the start button to begin capturing live network traffic on the chosen interface. Wireshark will start displaying packets in real time.

Capture only TCP traffic: tcp

Capture only UDP traffic: udp

Capture only HTTP traffic: tcp port 80

Capture only DNS traffic: udp port 53

Capture only ICMP traffic: icmp

Capture Only ARP traffic: arp

Capture DHCP traffic: port 67 or port 68

Step 5: While capturing, use display filters to isolate and analyze specific protocols more easily. Examples of display filters: TO display only TCP packets: tcp

TO display only UDP packets: udp

TO display only HTTP packets: http

to display only DNS packets: dns

TO display only ICMP packets: icmp

TO display only ARP packets: arp

TO display only DHCP packets: bootp (DHCP uses the BOOTP protocol)

Step 6: Let the capture run until you have collected enough packets for analysis, then click the Stop button to halt the capture process.

Step 7: Save the captured traffic by navigating to File > Save As.

Step 8: Review and Analyse.

Output:

The output will be a list of network packets displayed in wireshark's packet details window, which will include:

No: packet number in the capture sequence.

Time: Timestamp when the packet was captured.

Source: The Source IP address of the packet

Destination: The destination IP address of the packet.

Protocol: The protocol used (e.g TCP, UDP, HTTP, ARP, DNS).

Length: The size of the packet in bytes.

Info: Additional information about the packet (e.g., HTTP request/response, ARP request, ICMP echo

request)

Sample output:

NO.	Time	Source	Destination	protocol	Length	Info
1	0.00000	192.168.1.1	8.8.8.8	ICMP	56	Echo (ping) request
2	0.002345	8.8.8.8	192.168.1.1	ICMP	64	Echo (ping) Reply.
3	0.003234	192.168.1.2	192.168.1.1	ARP	42	Who has 192.168.1.1 Tell 192.168.1.2

Result:

Thus this experiment capture, save and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS using Wire Shark tool.

EXP: 15 ANALYZE THE DIFFERENT TYPES OF SERVERS USING WEBALIZE TOOL

AIM:

To Analyze the usage statistics of different types of servers (Apache, Nginx, FTP/proxy) using the webalizer tool.

PROCEDURE:

1. Install the webalizer tool in the system.
   Sudo apt-get update sudo
   apt-get install webalizer
   (On centOS: sudo yum install webalizer)
2. Verify installation:
   webalizer -V
3. Locate the log files of the servers:
   Apache: /var/log/apache2/access.log
   Nginx: /var/log/vsftpd.log or /var/log/squid/access.log
4. Run webalizer on the log file to generate a report:
   Sudo webalizer /var/log/apache2/access.log -o /var/www/html/webalizer/
5. Open a web browser and access the report
   http://localhost/webalizer/
6. Repeat the same process for Nginx, FTP, or proxy logs.
7. Compare and analyze the reports (hits, visits, bandwidth, top URLs, and countries).

Scanned with OKEN Scanner

PROGRAM:

#step 1: Install webalizer sudo apt-get install webalizer -y

#step 2: Run webalizer for Apache logs sudo webalizer /var/log/apache2/access.log -o /var/www/html/apache_report/

#step 3: Run webalizer for Nginx log sudo webalizer /var/log/nginx/access.log -o /var/www/html/nginx_report/

#step 4: Run webalizer for proxy logs sudo webalizer /var/log/squid/access.log -o /var/www/html/proxy_report/output.

Sample observation Table

Server
Type
Total
Hits
Unique
Visitors

Bandwidth
Used

Top
Client/Domain Report URL

Apache 12,560 1,240 1.8 GB 192.168.1.10
http://localhost/apache/apache_report/

9,840 980 1.2 GB 192.168.1.20
http://localhost/nginx_report/

Proxy 15,300 1,400 2.1 GB 192.168.1.30
http://localhost/proxy-report/

Graphical Output (as seen in browser):
monthly summary chart
Daily/hourly usage histogram
pie chart of top hosts and urls

RESULT:

The webalizer tool successfully generated graphical and statistical reports of different servers.
The Analysis provided insights into:
Total hits and visits Bandwidth usage hourly traffic

Ex:11 Examine Network Address Translation (NAT) using Cisco packet Tracer

Aim:

Implement Network Address Translation (NAT) using Cisco packet Tracer. We will Configure Static, dynamic, and Port Address Translation (PAT) on a router to enable communication between a private internal network and public external network.

Procedure:

Step 1: Setup the Network Topology

1. Open Cisco packet Tracer
2. Add devices:
   - 2 PCs (PC1 and PC2) representing inside Private network.
   - 1 Router representing the outside/ Public network.
3. Connect devices.
   - Connect PC1 and PC2 to the Switch.
   - Connect the Switch to the Routers GigabitEthernet 0/0 (Inside Interface).

Step 1: Open Cisco packet Tracer.

- Lauch the Cisco Packet Tracer Software.

Step 2: Add Devices.

1. Add PCs:
   - On the bottom device toolbar, click on End devices (the PC icon)
   - Drag and drop 2 PCs into the workspace.
2. Add a Switch (optional):
   - Click on Switches (looks like a small box with ports).
   - Drag and drop a 2960 switch into the workspace.
3. Add a Router:
   - Click on Routers.
   - Drag and drop a router with atleast two GigabitEthernet interfaces (e.g., 1941 router).
4. Add a cloud or Another Router (outside Network)
   - For Simplicity; Click on network devices → WAN Emulation → drag a Cloud.

Step 3: Connect Devices:

1. Connect PC1 and PC2 to the Switch
   - Click on the connections icon (lightning bolt)

Step 4: Verify Ip configuration

From PC1 or PC2, go to the Desktop tab → Command prompt.

Ping the router inside interface:

Ping 192.168.1.1

- From Router, ping the outside device: ping 200.1.1.2

Step : Open the router CLI and enter the following Command.

Code:

Step 4 > enable
configure
terminal
Interface Gigabit Ethernet 0/0 ip
address 192.168.1.1 255.255.255.0
no shutdown
exit
Interface Gigabit Ethernet 0/1 ip
Addres 200.1.1.1.255.255.255.0
no shutdown
exit
Interface Gigabit Ethernet 0/1 ip
inside exit interface
Gigabit Ethernet 0/1 ip nat

How to set default gateway on pc1 and Pc2

1. Click on Pc1 in the workspace
2. Go to Desktop tab.
3. Click on IP configuration
4. In the default gateway field type:
192.168.1.1

5. Make sure the IP address and subnet mask are already set.

You should see translation of private IPs to public IPs

6-2 Test Connectivity

- from pc1 or pc2, ping an IP on the outside network
- use packet tracer simulation mode to observe NAT translation on the packets as they leave the Router.

Result:

The router successfully translated private IP to Public ones using static/dynamic and PAT configurations.
All inside hosts could communicate with the external network, confirming that NAT

Ex.6 Building ... using FTPLIB Module

Aim:

To create a Python-based FTP scanner using the ftplib module that:

- checks if a public FTP server allows anonymous login.
- Lists the contents of directories on the server.

Procedure:

- Import the built in ftplib module.
- Write two python scripts:
  - Sender side: Connects to the FTP server, logs in anonymously, and lists the root directory.
- Run the scripts in Google Colab or any python environment.
- Observe the login status, server message, and directory contents.

code:

Sender side code:

```
#/usr/bin/env python3 import
ftplib
def anonymous login (hostname):
  try:
    ftp=ftplib.FTP(hostname)
anonymous login
Print(response)
if "230" in response: #login success code print
("(+)" + str(hostname) + "FTP Anonymous Login
Succeeded.")
    print("Welcome Message: \n",
          ftp.getwelcome())
    print("\n Directory listing :")
    ftp.dir() ftp.quit()
  else:
    Print("[-]" + str(hostname) + "FTP
Anonymous Login Failed.")
  anonymous Login (hostname)
```

Receiver side code:

```
import ftplib
def check_anonymous_connection(host,Path):
    try:
        with ftplib.FTP(host) as connection:
            connection.login('anonymous',
                             'anonymous')
            print("Welcome:",connection.gethelcom
                  ())
            print(f"\n Listing directory: {path}")

check_anonymous_connection(FTP_SERVER_
                           URL, DOWNLOAD_DIR_PATH
```

Scanned with OKEN Scanner

Output

Server side Output:

230 Anonymous access granted, restrictions apply [+]

ftp.be.debian.org FTP anonymous login Succeeded.

Welcome Message:

220 pro FTPD Server (mirror.as35701.net)

Directory Listing:

drwxr-xr-x 9 ftp ftp 4096

...

Receiver side output:

Welcome: 220 proFTPD Server (mirror.as35701.net)

Listing directory: /pub/linux/kernel/v5.x/

Linux-5.10.1.tar.xz file 113072564

linux-5.10.2.tar.xz file 113167444.

...

RESULT:

- The Script successfully connected to the FTP Server.
- It verified anonymous login is allowed
- The directory contents were retrieved
- displayed

8. SIMPLE CHATTING PROGRAM IN COMPUTER NETWORK

AIM:

To develop a simple chatting application using Python Socket programming to exchange messages b/w a client and Server.

Observation:

Chatting allows users to exchange messages through a computer network. Socket Programming provides communication b/w two computers using a client-server architecture.

TCP (Transmission control protocols) ensures reliable data transmission. Python's Socket Module is used to establish Communication, and the ending, module allows message to be received while typing

Procedure:

1) Install python and open VS code
2) create two files: Server.py and client.py
3) Import the Socket and threading modules

4) Create a
program 3.

5) Bind and start the server
then connect the client.

Program code:

→ Server program

```
Import Socket
Import threading
Server = Socket.Socket()
Server.Bind(("127.0.0.1", 12345))

While True:
    msg = input("Server: ")
    Conn.Send all(msg.encode(9)
    If msg.lower() == "exit":
        break

Conn.close()
Server.close()
```

~~client~~ Server:

```
Waiting for client
Client connected
Server: Hello
Client: Hi
Server: How are you?
Client: I am fine
```

Client:

```
Connected to server
Client: Hi
Server: Hello
Client: I am fine
Server: How are you?
```

Result:

Thus, a single chatting application was implement and successfully using python Socket programming and TCP to exchange messages b/w the client and server.

Scanned with OKEN Scanner